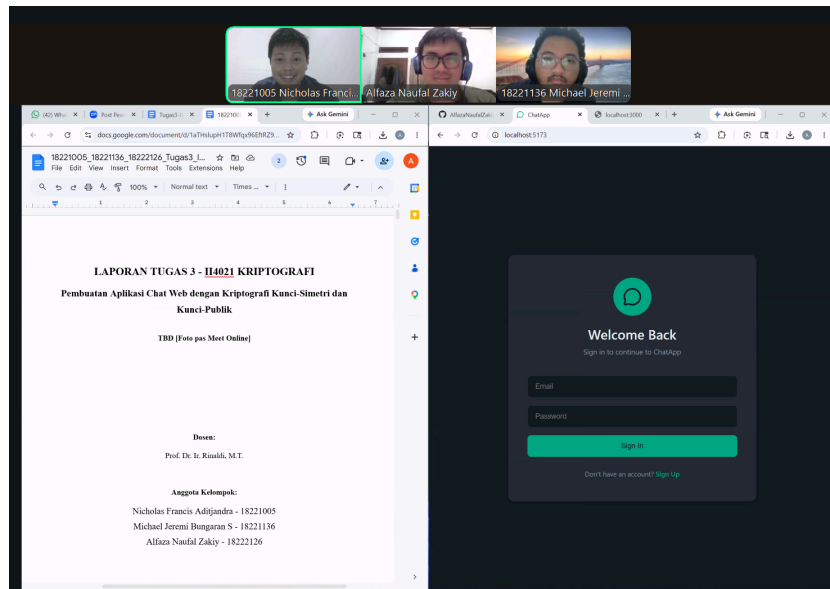


LAPORAN TUGAS 3 - II4021 KRIPTOGRAFI

Pembuatan Aplikasi Chat Web dengan Kriptografi Kunci-Simetri dan Kunci-Publik



Dosen:

Prof. Dr. Ir. Rinaldi, M.T.

Anggota Kelompok:

Nicholas Francis Aditjandra - 18221005

Michael Jeremi Bungaran S - 18221136

Alfaza Naufal Zakiy - 18222126

**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

2026

Daftar Isi

Daftar Isi	2
Pernyataan tidak melakukan kecurangan	4
1. Pendahuluan	5
2. Teknologi yang Digunakan	5
3. Teori Singkat	6
3.1. JSON Web Token	6
3.2. Elliptic Curve Diffie-Hellman (ECDH)	6
3.3. HMAC-based Key Derivation Function (HKDF)	6
3.4. AES-256-GCM	6
3.5. Message Authentication Code (MAC)	7
3.6. Web Crypto API	7
4. Perancangan Sistem	7
4.1. Arsitektur Umum	7
4.2. Struktur Database	9
5. Implementasi	9
5.1. Registrasi Pengguna	9
5.2. Login dan Autentikasi JWT	10
5.3. Daftar Kontak	11
5.4. Pembentukan Kunci Komunikasi	11
5.5. Pengiriman dan Penerimaan Pesan	11
5.6. Bonus: Integritas dan Autentikasi Pesan	12
6. Pengujian dan Analisis Hasil	13
6.1. Uji Autentikasi dan Manajemen Pengguna	13
6.2. Uji Enkripsi dan Dekripsi Pesan	16
6.3. Uji Bonus MAC	17
7. Cara Menjalankan Program	20
7.1. Backend	20
7.2. Frontend	20
7.3. Verifikasi Build	20
8. Kesimpulan	20
Daftar Pustaka	22
Lampiran	23
Pranala Repositori Perangkat Lunak	23
Pranala Video Demo	23
Pembagian Tugas	23
1. Antar Muka Program	24

Pernyataan tidak melakukan kecurangan

Kami menyatakan bahwa kode program yang dihasilkan bukan merupakan hasil salinan mentah (*raw output*) dari *Generative AI*, melainkan hasil pengembangan dan penulisan mandiri.



Nicholas Francis Aditjandra



Michael Jeremi Bungaran S



Alfaza Naufal Zakiy

1. Pendahuluan

Aplikasi chat berbasis web membutuhkan mekanisme keamanan yang tidak hanya mengatur akses pengguna, tetapi juga menjaga kerahasiaan dan integritas pesan. Pada tugas ini, dikembangkan aplikasi chat yang menggabungkan autentikasi berbasis JSON Web Token (JWT), pembentukan kunci komunikasi dengan Elliptic Curve Diffie-Hellman (ECDH), derivasi kunci dengan HKDF, enkripsi pesan dengan AES-256, serta verifikasi integritas pesan dengan Message Authentication Code (MAC).

Server berperan sebagai penyimpan data pengguna dan perantara pesan terenkripsi. Operasi kriptografi untuk komunikasi antar pengguna dilakukan di sisi klien menggunakan Web Crypto API, sehingga server tidak mengetahui plaintext pesan maupun kunci simetris percakapan.

2. Teknologi yang Digunakan

Komponen	Teknologi
Frontend	React, TypeScript, Vite
Backend	Go, Fiber
Database	SQLite
HTTP Client	Axios
Kriptografi sisi klien	Web Crypto API
Hash password	bcrypt dengan salt unik
JWT backend	Library lokal pada <code>backend/pkg/jwtlib</code> dan ECDSA ES256

Dependensi utama frontend terdapat pada `frontend/package.json`, sedangkan dependensi backend terdapat pada `backend/go.mod`.

3. Teori Singkat

3.1. JSON Web Token

JSON Web Token (JWT) adalah format token berbasis JSON yang digunakan untuk membawa klaim autentikasi. Pada implementasi ini, JWT digunakan sebagai bukti bahwa pengguna sudah login. Token diterbitkan backend setelah password berhasil diverifikasi. JWT menggunakan format JWS, yaitu token yang terdiri dari header, payload, dan signature:

```
base64url(header).base64url(payload).base64url(signature)
```

Algoritma yang digunakan adalah ES256, yaitu ECDSA dengan kurva P-256 dan SHA-256. Implementasi fungsi **Sign** dan **Verify** dibuat pada backend/pkg/jwtlib/jwt.go.

3.2. Elliptic Curve Diffie-Hellman (ECDH)

ECDH adalah protokol pertukaran kunci yang memungkinkan dua pihak menghasilkan shared secret yang sama menggunakan private key milik sendiri dan public key milik pihak lawan. Pada aplikasi ini, setiap pengguna memiliki pasangan kunci ECDH P-256 yang dibangkitkan di browser. Public key dikirim ke server, sedangkan private key dienkripsi terlebih dahulu sebelum disimpan.

3.3. HMAC-based Key Derivation Function (HKDF)

HKDF adalah fungsi derivasi kunci berbasis HMAC. Shared secret hasil ECDH tidak langsung digunakan sebagai kunci AES. Shared secret diproses dengan HKDF untuk menghasilkan kunci simetris yang sesuai. Pada aplikasi ini, HKDF menghasilkan dua kunci berbeda:

- Kunci AES-256-GCM untuk enkripsi dan dekripsi pesan.
- Kunci HMAC-SHA-256 untuk integritas pesan.

3.4. AES-256-GCM

AES adalah algoritma enkripsi simetris. Implementasi ini menggunakan AES-GCM dengan panjang kunci 256 bit. AES-GCM membutuhkan IV unik untuk setiap enkripsi. Ciphertext dan IV disimpan di server sebagai bagian dari payload pesan.

3.5. Message Authentication Code (MAC)

MAC digunakan untuk memverifikasi integritas dan autentikasi pesan. Pada aplikasi ini, MAC dihitung dengan HMAC-SHA-256 menggunakan kunci hasil HKDF. Data yang dilindungi MAC meliputi `senderEmail`, `receiverEmail`, `iv`, dan `ciphertext`. Jika MAC tidak sesuai, pesan ditandai tidak valid dan tidak didekripsi.

3.6. Web Crypto API

Web Cryptography API menyediakan fondasi kriptografi tingkat rendah di browser. Fungsinya mencakup enkripsi, dekripsi, hashing data, tanda tangan digital, serta membuat, mengimpor, mengeksport, dan menurunkan kunci. Tujuan utamanya yaitu aplikasi web yang mampu mengelola data sensitif secara aman di sisi klien, tanpa harus mengekspos atau mengirim kunci mentah ke server.

4. Perancangan Sistem

4.1. Arsitektur Umum

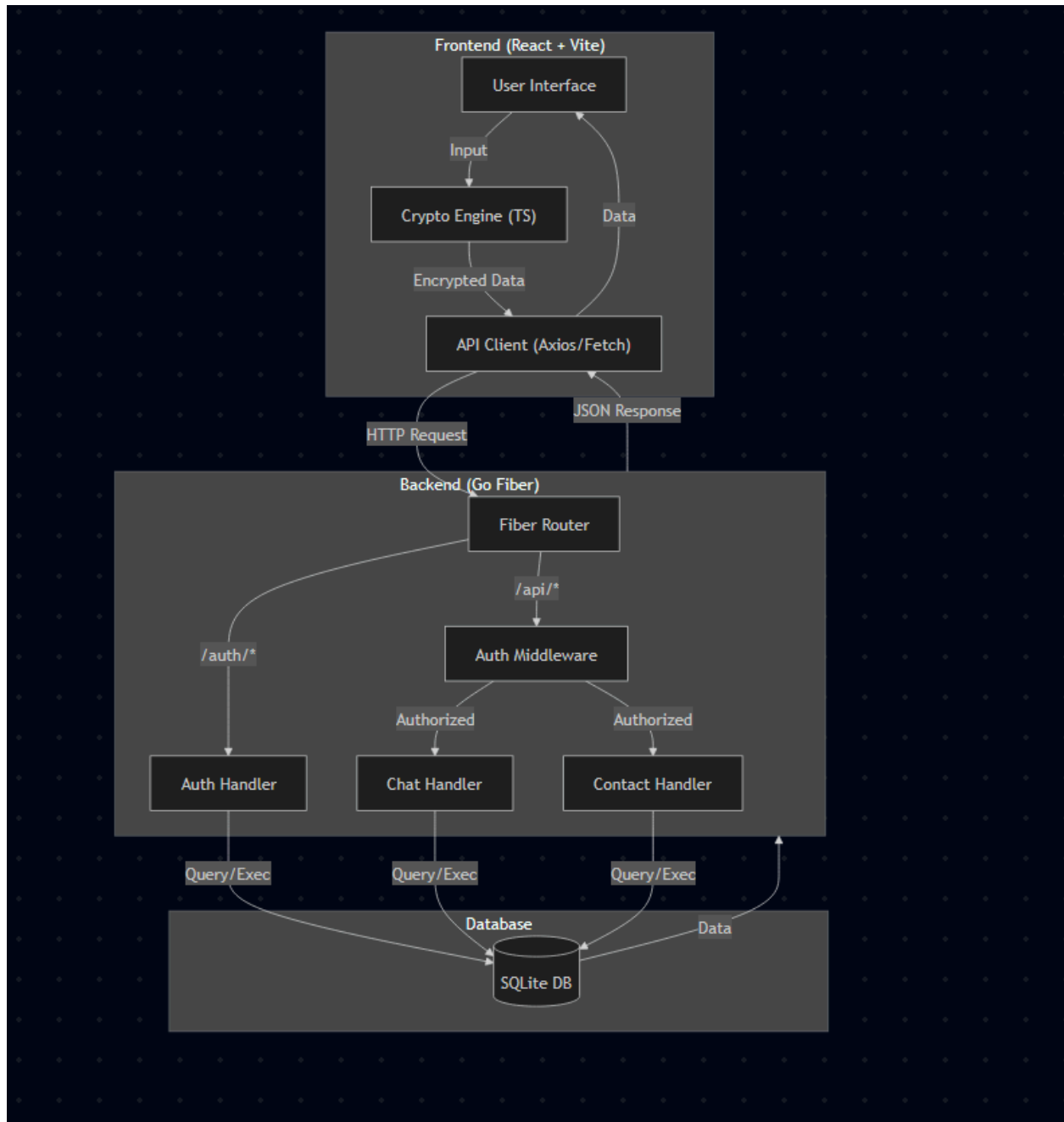
Aplikasi terdiri dari client web, server backend, dan database SQLite. Client bertanggung jawab untuk operasi kriptografi komunikasi, sedangkan server bertanggung jawab untuk autentikasi, penyimpanan public key, penyimpanan private key terenkripsi, dan penyimpanan pesan terenkripsi.

Alur umum sistem:

1. Pengguna melakukan registrasi dengan email dan password.
2. Browser membangkitkan pasangan kunci ECDH P-256.
3. Private key ECDH dienkripsi menggunakan AES-256-GCM dengan kunci dari PBKDF2 password.
4. Server menyimpan email, hash password, salt password, public key, private key terenkripsi, salt KDF, dan IV private key.
5. Pengguna login dengan email dan password.
6. Server memverifikasi password dan menerbitkan JWT ES256.
7. Browser mendekripsi private key pengguna menggunakan password.
8. Ketika pengguna memilih kontak, browser mengambil public key kontak dari server.
9. Browser membentuk shared secret dengan ECDH dan menurunkan kunci AES/HMAC dengan HKDF.

10. Pesan dienkripsi dengan AES-256-GCM dan diberi MAC sebelum dikirim ke server.
11. Penerima memverifikasi MAC dan mendekripsi pesan di sisi browser.

Diagram Sistem :



4.2. Struktur Database

Skema database dibuat pada backend/internal/database/db.go.

Tabel users menyimpan:

- email
- password_hash
- password_salt
- public_key
- encrypted_private_key
- kdf_salt
- private_key_iv

Tabel messages menyimpan:

- sender_email
- receiver_email
- ciphertext
- iv
- mac
- timestamp

Server tidak menyimpan plaintext pesan maupun private key plaintext.

5. Implementasi

5.1. Registrasi Pengguna

Implementasi registrasi frontend terdapat pada:

- frontend/src/components/Auth/Register.tsx
- frontend/src/utls/crypto.ts
- frontend/src/utls/api.ts

Pada registrasi, browser menjalankan `generateRegistrationKeys(password)` di `frontend/src/utls/crypto.ts`. Fungsi ini melakukan:

1. Generate key pair ECDH P-256 dengan Web Crypto API.
2. Export public key dan private key dalam format JWK.
3. Membuat salt PBKDF2 dan IV AES-GCM secara acak.
4. Menurunkan kunci enkripsi private key dari password menggunakan PBKDF2 SHA-256.
5. Mengenkripsi private key dengan AES-256-GCM.
6. Mengirim public key, encrypted private key, salt, dan IV ke server.

Implementasi backend terdapat pada `backend/internal/handlers/auth.go`. Backend melakukan:

1. Validasi field registrasi.
2. Mengecek apakah email sudah terdaftar.
3. Membuat salt password acak.
4. Membuat hash password dengan `bcrypt` dari `password_salt + password`.
5. Menyimpan data pengguna ke SQLite.

Jika email sudah digunakan, backend mengembalikan:
`This email address is already registered.`

5.2. Login dan Autentikasi JWT

Implementasi login frontend terdapat pada `frontend/src/components/Auth/Login.tsx`. Pengguna memasukkan email dan password. Frontend mengirim kredensial ke endpoint `/auth/login`.

Implementasi login backend terdapat pada `backend/internal/handlers/auth.go`. Backend mengambil data pengguna dari database, memverifikasi password dengan `bcrypt`, lalu menerbitkan JWT.

JWT dibuat dengan fungsi `Sign` pada `backend/pkg/jwtlib/jwt.go`. Header JWT:

```
{
  "alg": "ES256",
  "typ": "JWT"
}
```

Klaim yang digunakan:

- `iss` : issuer aplikasi
- `sub` : email pengguna
- `aud` : audience aplikasi
- `iat` : waktu token diterbitkan
- `nbf` : waktu mulai berlaku
- `exp` : waktu kedaluwarsa
- `jti` : ID token acak
- `email` : email pengguna

Verifikasi JWT dilakukan pada middleware `backend/internal/middleware/auth.go` dengan fungsi `Verify` dari `backend/pkg/jwtlib/jwt.go`. Middleware ini melindungi endpoint `/api`.

Kunci ECDSA untuk JWT dibuat saat backend berjalan pada `backend/internal/authkeys/keys.go`

5.3. Daftar Kontak

Daftar kontak diambil dari endpoint `/api/contacts`. Implementasinya terdapat pada `backend/internal/handlers/contacts.go`. Endpoint ini mengembalikan semua pengguna yang sudah terdaftar kecuali pengguna yang sedang login.

Di frontend, kontak dipetakan pada `frontend/src/App.tsx` dan ditampilkan melalui komponen sidebar.

5.4. Pembentukan Kunci Komunikasi

Pembentukan kunci komunikasi dilakukan di sisi klien pada `frontend/src/utils/crypto.ts`, terutama fungsi `deriveConversationKeys`.

Langkahnya:

1. Import public key kontak dari format JWK.
2. Gunakan private key pengguna dan public key kontak untuk ECDH.
3. Hasil ECDH diproses dengan HKDF SHA-256.
4. HKDF menghasilkan kunci AES-256-GCM dengan info `ii4021-chat-aes`.
5. HKDF menghasilkan kunci HMAC-SHA-256 dengan info `ii4021-chat-mac`.

Salt HKDF dibentuk dari pasangan email yang diurutkan agar kedua pihak menggunakan konteks derivasi yang sama. Info dipakai agar dapat diketahui kunci mana yang digunakan untuk dekripsi dan kunci mana yang digunakan untuk pencocokan.

5.5. Pengiriman dan Penerimaan Pesan

Pengiriman pesan diatur pada `frontend/src/components/Chat/ChatArea.tsx`.

Ketika pengguna mengirim pesan:

1. Browser membentuk kunci percakapan.
2. Plaintext dienkripsi dengan AES-256-GCM.
3. Browser menghitung HMAC-SHA-256 atas metadata dan ciphertext.
4. Payload berisi `sender_email`, `receiver_email`, `ciphertext`, `iv`, `mac`, dan `timestamp` dikirim ke server.

Backend menerima payload pada `backend/internal/handlers/chat.go`. Server hanya memvalidasi bahwa pengirim sesuai dengan JWT dan field terenkripsi lengkap, lalu menyimpan payload ke database.

Penerimaan pesan dilakukan pada `frontend/src/components/Chat/ChatArea.tsx`. Pesan dari server diverifikasi dan didekripsi dengan `decryptMessage` di `frontend/src/utlis/crypto.ts`. Jika MAC tidak valid atau dekripsi gagal, UI menampilkan:
`Message could not be decrypted`

Penanda error ditampilkan pada `frontend/src/components/Chat/MessageBubble.tsx`.

5.6. Bonus: Integritas dan Autentikasi Pesan

Bonus MAC diimplementasikan pada `frontend/src/utlis/crypto.ts`. MAC menggunakan HMAC-SHA-256 dengan kunci terpisah dari kunci AES. Data yang dimasukkan ke MAC:

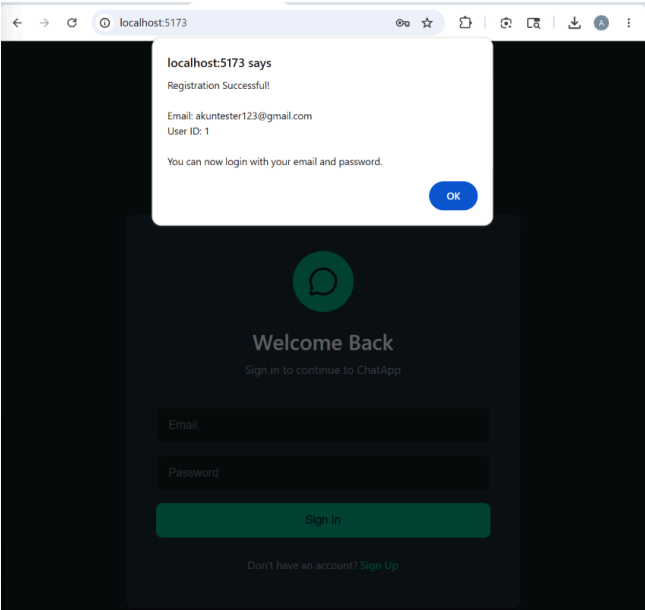
```
senderEmail.receiverEmail.iv.ciphertext
```

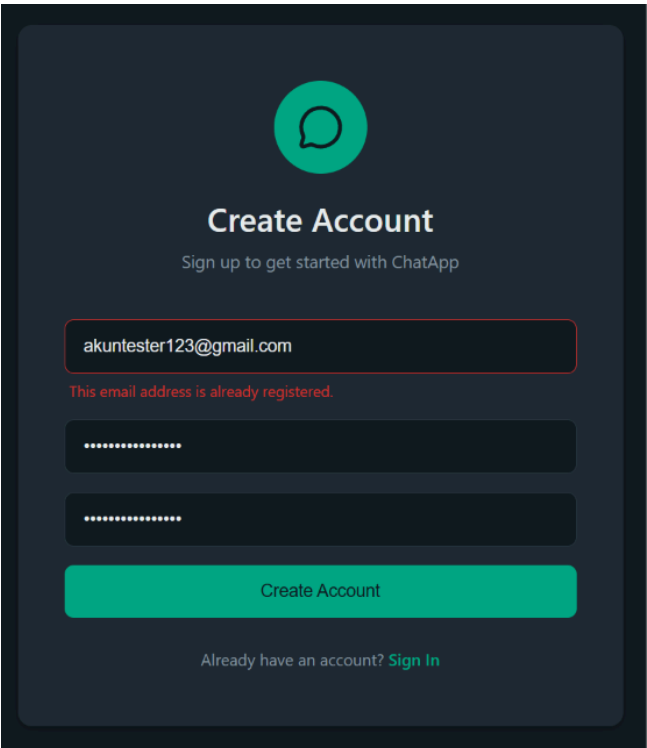
Dengan memasukkan metadata pengirim dan penerima, perubahan pada arah pesan atau data ciphertext dapat terdeteksi oleh penerima. Verifikasi dilakukan sebelum proses dekripsi.

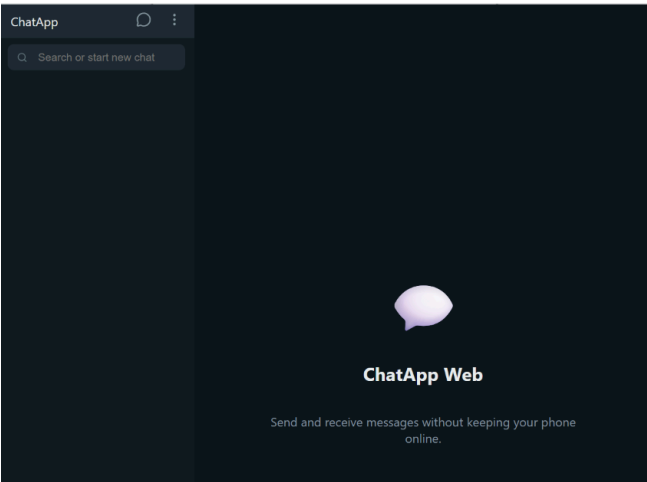
6. Pengujian dan Analisis Hasil

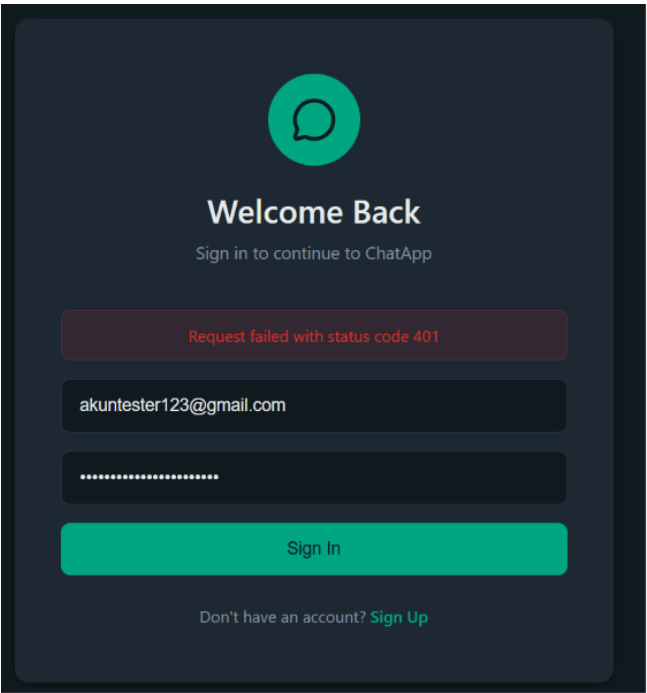
Bagian ini berisi rancangan pengujian yang sesuai dengan spesifikasi. Pada laporan PDF final, setiap skenario sebaiknya dilengkapi screenshot antarmuka, respons API, atau isi database.

6.1. Uji Autentikasi dan Manajemen Pengguna

No	Kasus Uji	Langkah	Hasil yang Diharapkan	Status
1	Registrasi data valid	Daftar dengan email baru dan password valid	Akun berhasil dibuat, data pengguna masuk ke tabel <code>users</code> , password tersimpan sebagai hash, private key tersimpan terenkripsi	 A screenshot of a web browser window at localhost:5173. A white notification box at the top says 'localhost:5173 says Registration Successful! Email: akuntester123@gmail.com User ID: 1. You can now login with your email and password.' with an 'OK' button. Below the notification is a dark-themed login form with a green speech bubble icon, the text 'Welcome Back', and 'Sign in to continue to ChatApp'. It has input fields for 'Email' and 'Password', a green 'Sign In' button, and a link 'Don't have an account? Sign Up'.

2	Registrasi email sudah terdaftar	Daftar ulang dengan email yang sama	Sistem menolak registrasi dan menampilkan This email address is already registered.	
---	----------------------------------	-------------------------------------	---	---

3	Login password benar	Login dengan email dan password benar	Backend menerbitkan JWT dan frontend berhasil masuk ke halaman chat	 A screenshot of a web browser displaying the ChatApp Web interface. The browser's address bar shows 'localhost:5173'. The page has a dark theme. On the left, there is a sidebar with a search bar labeled 'Search or start new chat'. The main area features a large purple speech bubble icon, the text 'ChatApp Web', and a subtitle 'Send and receive messages without keeping your phone online.'
---	----------------------	---------------------------------------	---	--

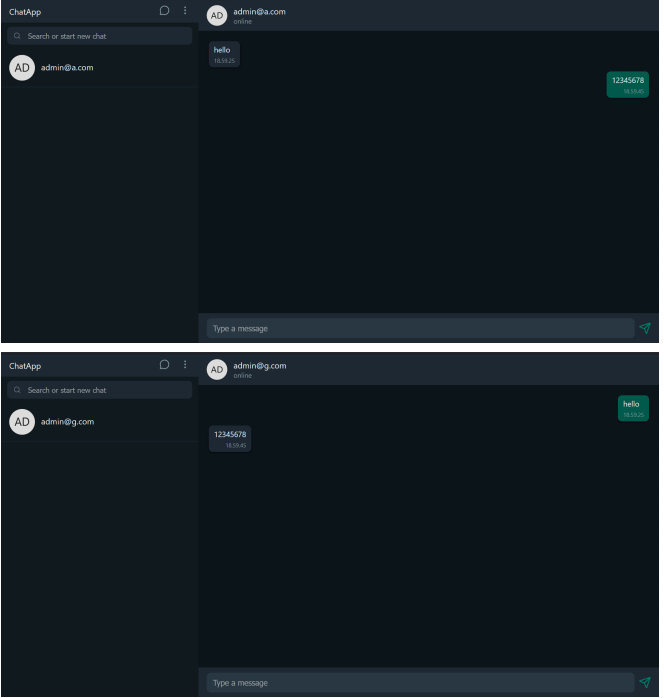
4	Login password salah	Login dengan password salah	Sistem menolak login dengan pesan Request failed with status code 401	
---	----------------------	-----------------------------	--	---

Analisis:

Password tidak disimpan dalam plaintext karena backend menyimpan `password_hash` dan `password_salt`. Private key juga tidak disimpan dalam plaintext karena browser mengenkripsi private key sebelum dikirim ke server.

6.2. Uji Enkripsi dan Dekripsi Pesan

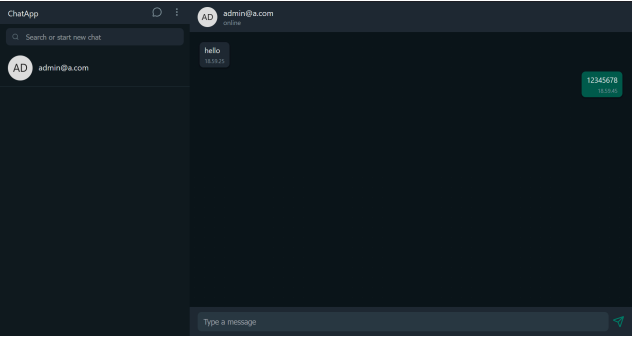
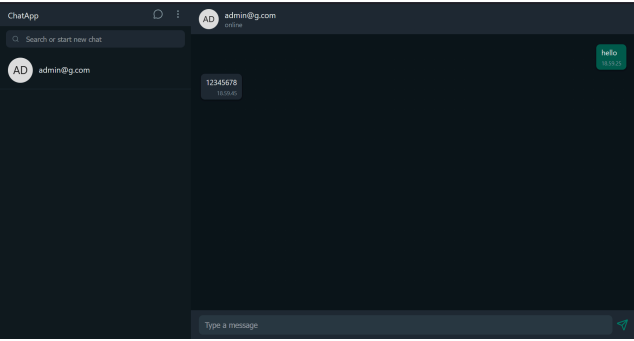
No	Kasus Uji	Langkah	Hasil yang Diharapkan	Status
----	-----------	---------	-----------------------	--------

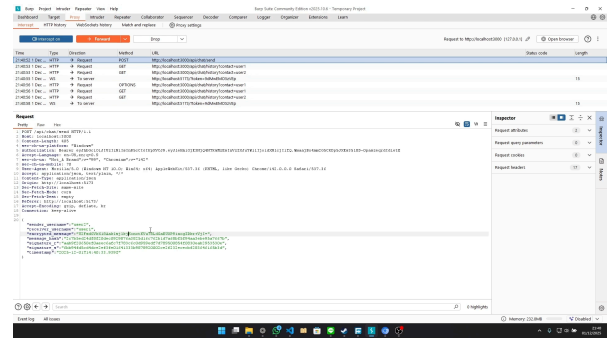
1	Kirim pesan dengan kunci benar	User A mengirim pesan ke User B	Server menyimpan ciphertext, IV, dan MAC; User B melihat plaintext setelah dekripsi	
---	--------------------------------	---------------------------------	---	---

Analisis:

Server hanya melihat ciphertext. Plaintext hanya muncul di browser pengguna yang berhasil membentuk kunci percakapan yang sama.

6.3. Uji Bonus MAC

No	Kasus Uji	Langkah	Hasil yang Diharapkan	Status
1	MAC valid	Kirim pesan normal tanpa perubahan payload	MAC valid dan pesan diproses/didekripsi	 

2	Payload diubah	Ubah sebagian ciphertext, IV, sender, receiver, atau MAC pada database/response	Verifikasi MAC gagal dan pesan ditandai tidak valid sebelum dekripsi	 (Pesan tampil kosong)
---	----------------	---	--	--

Analisis:

Metadata ikut dilindungi MAC, sehingga perubahan identitas pengirim/penerima atau isi pesan dapat dideteksi penerima.

7. Cara Menjalankan Program

7.1. Backend

```
cd backend
go mod tidy
go run main.go
```

Backend berjalan pada:
`http://localhost:3000`

7.2. Frontend

```
cd frontend
npm install
npm run dev
```

Frontend Vite biasanya berjalan pada:
`http://localhost:5173`

7.3. Verifikasi Build

Perintah verifikasi:

```
cd frontend
npm run build
```

```
cd backend
go test ./...
```

8. Kesimpulan

Aplikasi berhasil menerapkan alur utama secure chat sesuai spesifikasi. Registrasi dan login dilakukan dengan email dan password, password disimpan dalam bentuk hash dengan salt, JWT diterbitkan dan diverifikasi menggunakan ECDSA ES256, serta komunikasi antar pengguna dilindungi dengan ECDH, HKDF, AES-256-GCM, dan HMAC-SHA-256. Seluruh operasi kriptografi komunikasi dilakukan di sisi klien menggunakan Web Crypto API.

Server tidak memegang private key plaintext, shared secret, kunci AES, maupun plaintext pesan. Server hanya menyimpan data pengguna, public key, private key terenkripsi, ciphertext, IV, MAC, dan metadata pesan.

Tugas bonus yang telah di implementasi adalah integritas pesan dengan MAC.

Daftar Pustaka

- IETF. RFC 7519: JSON Web Token (JWT).
- IETF. RFC 7515: JSON Web Signature (JWS).
- MDN Web Docs. Web Crypto API.
- NIST. Advanced Encryption Standard (AES).
- IETF. RFC 5869: HMAC-based Extract-and-Expand Key Derivation Function (HKDF).
- Provos, N. dan Mazieres, D. A Future-Adaptable Password Scheme. USENIX, 1999.
- Arsinio, D. (2025, 25 November). Web Crypto: Pengertian, Kegunaan, dan Cara Memanfaatkannya. Pluang Akademi.
<https://pluang.com/akademi/berita-analisis/web-crypto-251125>
- Munir, R. (2025). [a\) Elliptic Curve Cryptography \(ECC\) - Bagian 1](#) (Update materi 2025). IF4020 Kriptografi, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.
- Munir, R. (2025). [a\) Elliptic Curve Cryptography \(ECC\) - Bagian 2](#) (Update materi 2025). IF4020 Kriptografi, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.
- Munir, R. (2025). [Fungsi hash kriptografi](#) (Update materi 2025). IF4020 Kriptografi, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.
- Munir, R. (2025). [Beberapa fungsi hash \(MD5, SHA-1, SHA-256, SHA-3/Keccak\)](#) (Update materi 2025). IF4020 Kriptografi, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.
- Munir, R. (2025). [Digital Signature Algorithm \(DSA\) dan Elliptic Curve Digital Signature Algorithm \(ECDSA\) \(Update materi 2025\)](#). IF4020 Kriptografi, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

Lampiran

Pranala Repositori Perangkat Lunak

<https://github.com/AlfazaNaufalZakiy/Tugas-3-Aplikasi-Chat-Web-dengan-Kunci-Simetri-dan-Kunci-Publik-Kriptografi-STI>

Pranala Video Demo

<https://youtu.be/N567MgYoDfM>

Pembagian Tugas

Nama Lengkap	NIM	Deskripsi Tugas
Nicholas Francis Aditjandra	18221005	- Mengerjakan bagian frontend aplikasi chat - Mengerjakan bagian komunikasi frontend dan backend - Mengerjakan laporan - Melakukan pengujian
Michael Jeremi Bungaran S	18221136	- Mengerjakan fungsi kriptografi dan pengecekan integrasi - Mengerjakan laporan
Alfaza Naufal Zakiy	18222126	- Mengerjakan bagian autentikasi - Mengerjakan bagian pengiriman pesan - Mengerjakan laporan